



AeroTrack
Air quality monitoring with Apache Hop

RELATÓRIO DE DESENVOLVIMENTO

Monitoramento de Qualidade do Ar com Apache Hop

Trabalho Final | Módulo 6 | Apache Hop



Equipe AeroTrack

Allef Oliveira Ramos	@allef-oliveira
Fernanda de Oliveira da Costa	@nanda-costa
Juliana Ballin Lima	@JulianaBallin
Pedro Henrique Oliveira Dias	@pedroddias-oss

Manaus | 2026

Capacitação em IA e Transformação Digital



Identificação do documento

Projeto	AeroTrack
Documento	Relatório de Desenvolvimento
Curso	Capacitação em IA e Transformação Digital
Módulo	Módulo 6 de Apache Hop
Projeto sugerido	Projeto 4, Monitoramento de Qualidade do Ar
Repositorio	https://github.com/JulianaBallin/AeroTrack
Branch de desenvolvimento	develop
Versão	1.0
Situação	Pipelines, workflow, banco de dados e dashboard validados de ponta a ponta

Controle de revisão

Este documento registra o problema, a arquitetura, as pipelines, o tratamento de dados, o banco de dados, os indicadores, o dashboard e a validação dos resultados do AeroTrack. Todas as execuções descritas foram realizadas contra o banco de dados real, via linha de comando do Apache Hop.



Resumo executivo

O AeroTrack é uma solução de integração de dados construída com Apache Hop para o Projeto 4 sugerido no trabalho final do Módulo 6: monitoramento de qualidade do ar. O dataset público Air Quality (UCI Machine Learning Repository) é lido, tratado, classificado e persistido em um banco de dados PostgreSQL, com indicadores consolidados e apresentados em um dashboard.

Resultado técnico atual

- Duas pipelines Apache Hop (ingestão/tratamento e consolidação) e um workflow de orquestração, executados de ponta a ponta sem erros.
- 9357 leituras horárias processadas: 8131 gravadas como válidas e 1226 separadas por não terem nenhum poluente de referência.
- Banco de dados PostgreSQL com quatro tabelas e duas visões, reexecutável sem duplicidade por chave única e Insert/Update.
- Indicadores diários (363 dias) e mensais (14 meses) consolidados.
- Dashboard estático com sete indicadores, quatro gráficos e o ranking dos dez piores dias.



Sumário

Identificação do documento	i
Resumo executivo	ii
1 Contexto, objetivo e problema	1
1.1 Contexto acadêmico	1
1.2 Objetivo geral	1
1.3 Dataset utilizado	1
1.4 Problema principal	2
2 Arquitetura da solução	3
2.1 Visão geral	3
2.2 Tecnologias utilizadas	3
3 Pipelines Apache Hop	4
3.1 pl_01_ingestion_treatment	4
3.2 pl_02_indicator_consolidation	6
3.3 Workflow de orquestração	7
4 Tratamento de dados	9
4.1 Valor sentinela -200	9
4.2 Linhas vazias do export	9
4.3 Separação de leituras inválidas	9
4.4 Classificação analítica	9
5 Banco de dados	10



5.1	Estrutura de tabelas	10
5.2	Reexecução e controle de duplicidade	11
6	Indicadores e dashboard	12
6.1	Indicadores calculados	12
6.2	Dashboard	12
7	Validação dos resultados	15
7.1	Execução do workflow completo	15
7.2	Indicadores gerais obtidos	15
7.3	Conferência de contagens	16
8	Conclusão	17
8.1	Principal resultado	17
8.2	Principais decisões técnicas	17
8.3	Aprendizado	17
9	Manual de replicação	18
9.1	Pré-requisitos	18
9.2	Passo a passo	18
9.3	Solução de problemas	19
10	Equipe	20



Contexto, objetivo e problema

1.1 Contexto acadêmico

O trabalho final do Módulo 6 pede uma solução prática de ETL com Apache Hop, partindo de um dataset público, com tratamento de dados dentro da pipeline, persistência em banco relacional, orquestração por workflow, indicadores e um dashboard.

1.2 Objetivo geral

Demonstrar, de forma prática, o uso do Apache Hop para transformar dados brutos de qualidade do ar em informação estruturada, apoiando a análise de períodos com piores condições ambientais.

1.3 Dataset utilizado

Dataset	Air Quality
Fonte	UCI Machine Learning Repository, https://archive.ics.uci.edu/dataset/360/air+quality
Origem dos dados	Estação de monitoramento em uma cidade italiana (De Vito et al.)
Período	Marco de 2004 a fevereiro de 2005
Volume	9357 leituras horárias, 15 variáveis



1.4 Problema principal

Em quais períodos ocorreram as piores condições de qualidade do ar registradas, e quais variáveis ambientais (temperatura e umidade) estavam associadas a esses períodos? O ranking dos dez dias com maior média de CO, cruzado com a média de NO2 e o percentual de horas em condição crítica, responde diretamente a essa pergunta.



Arquitetura da solução

2.1 Visão geral



Figura 2.1: Fluxo de ponta a ponta do AeroTrack: do dataset público ao dashboard.

As duas pipelines são orquestradas pelo workflow `wf_air_quality_orchestration.hwf`: a consolidação só é executada após o sucesso da ingestão.

2.2 Tecnologias utilizadas

- **Apache Hop 2.18**: pipelines, workflow e metadados de conexão.
- **PostgreSQL 16**: banco de dados relacional, em container Docker.
- **Docker Compose**: subida do banco de dados local.
- **Chart.js**, vendorizado localmente: gráficos do dashboard.
- **Python 3 e psql**: exportação dos indicadores para o dashboard.



Pipelines Apache Hop

O projeto `aerotrack` é aberto no Apache Hop Designer com dois pipelines e um workflow de orquestração, listados na árvore do projeto.

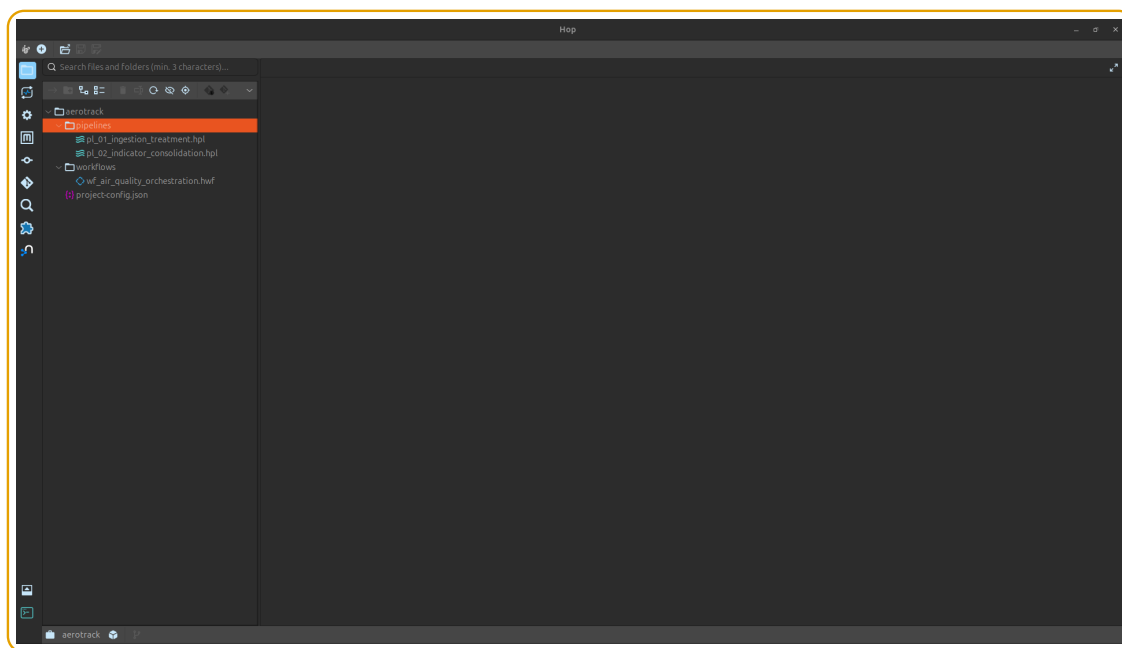


Figura 3.1: Hop Designer com o projeto `aerotrack` aberto e a árvore de pipelines e workflows visível. Situação atual: disponível.

3.1 `pl_01_ingestion_treatment`

Etapa	Transformação	O que faz
Ingestão	CSV Input	Le <code>data/raw/AirQualityUCI.csv</code> (separador <code>;</code> , decimal <code>,</code>)



Etapa	Transformação	O que faz
Qualidade	Filter Rows	Remove as linhas vazias do final do export do dataset
Tratamento	Formula	Combina data e hora originais em um texto único
Tratamento	Null If	Converte o sentinela -200 em nulo, em 13 campos numéricos
Tratamento	Select Values	Renomeia os campos tratados e remove os campos auxiliares
Tratamento	Select Values	Converte o texto de data e hora para o tipo Data
Enriquecimento	Calculator	Deriva ano, mes, dia, hora, dia da semana e a data de referência
Enriquecimento	Value Mapper	Converte o número do dia da semana no nome por extenso
Enriquecimento	Number Range	Classifica a hora em turno do dia
Enriquecimento	Number Range	Classifica o nível de CO
Enriquecimento	Number Range	Classifica o nível de NO2
Regra de negócio	Janino	Calcula se a leitura é válida e se a condição é crítica
Qualidade	Filter Rows	Separa leituras válidas das leituras sem poluente de referência
Banco de dados	Insert/Update	Grava as leituras válidas em leituras_qualidade_ar
Banco de dados	Table Output	Grava as leituras rejeitadas em leituras_rejeitadas

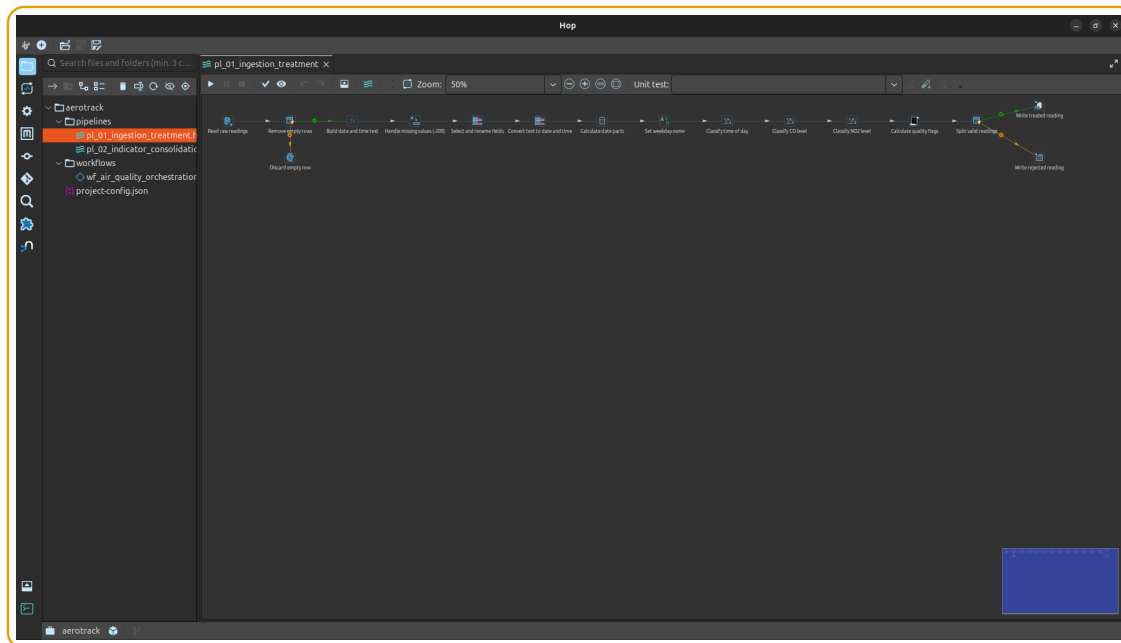


Figura 3.2: Canvas de `pl_01_ingestion_treatment` no Hop Designer. Situação atual: disponível.

3.2 `pl_02_indicator_consolidation`

Etapa	Transformação	O que faz
Consolidação diária	Table Input	Agrega leituras_qualidade_ar por data
Banco de dados	Insert/Update	Grava o resumo em <code>resumo_diario_qualidade_ar</code>
Consolidação mensal	Table Input	Agrega leituras_qualidade_ar por ano e mes
Banco de dados	Insert/Update	Grava o resumo em <code>resumo_mensal_qualidade_ar</code>

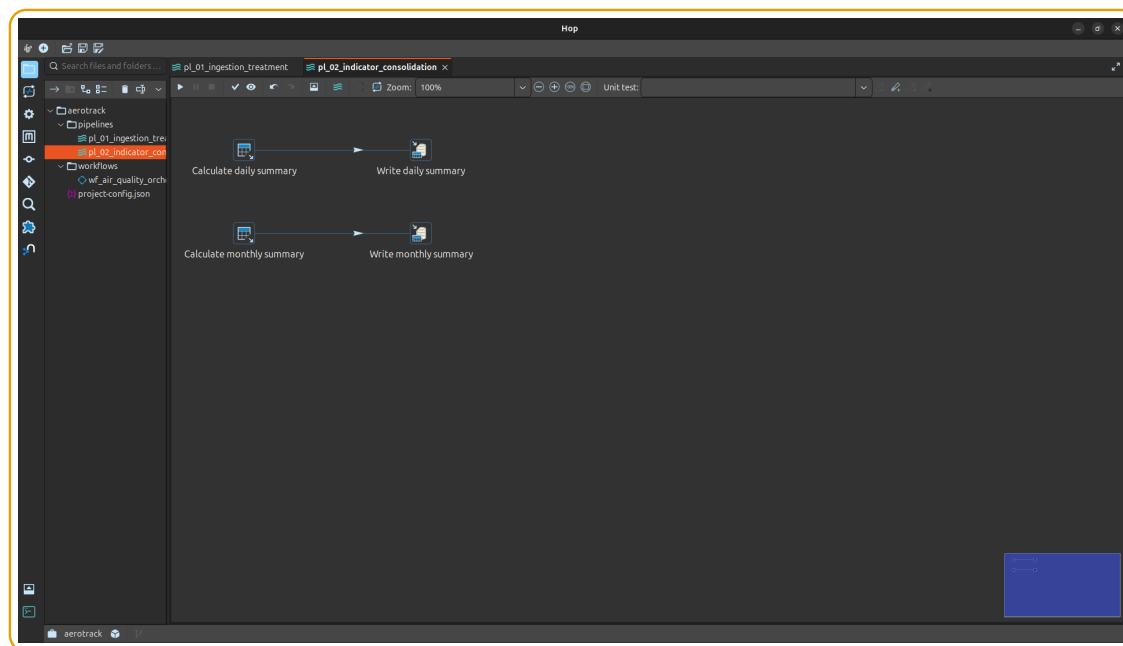


Figura 3.3: Canvas de pl_02_indicator_consolidation no Hop Designer. Situação atual: disponível.

3.3 Workflow de orquestração

```
START -> Ingestion and treatment -> Indicator consolidation -> Process completed
```

O segundo hop só é seguido se a pipeline de ingestão terminar com sucesso.

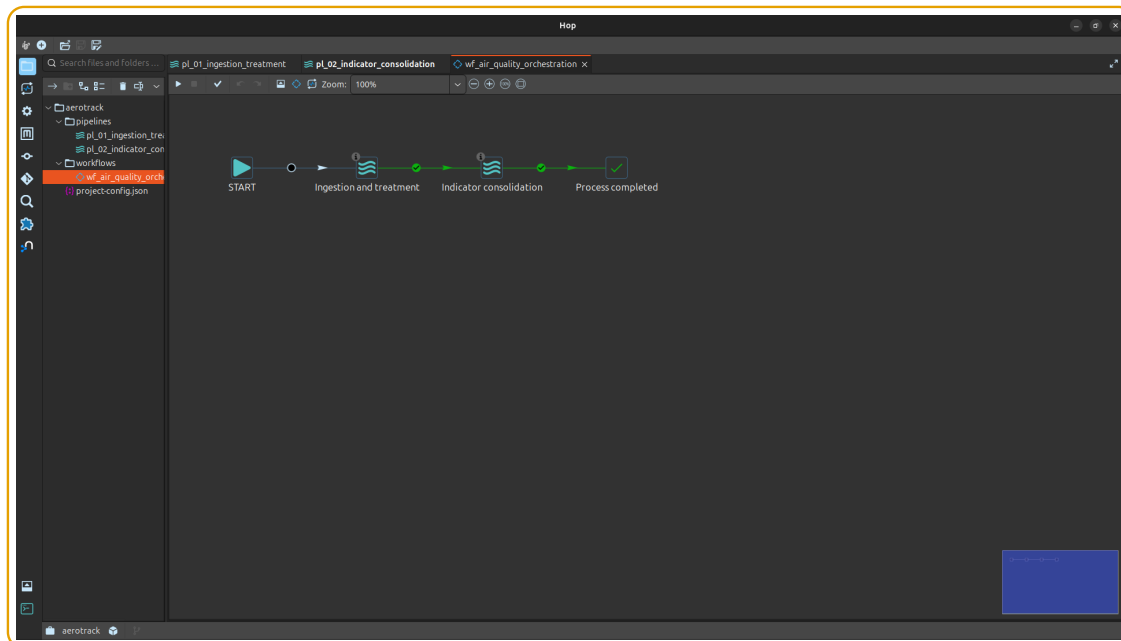


Figura 3.4: Canvas de wf_air_quality_orchestration no Hop Designer. Situação atual: disponível.

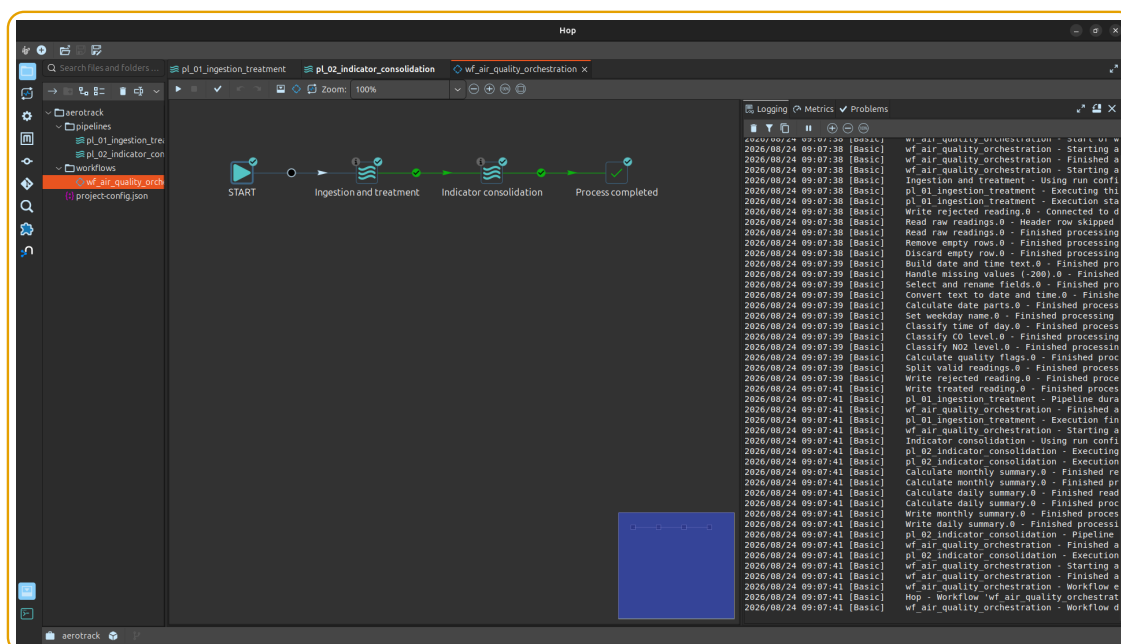


Figura 3.5: Execução do workflow pela interface gráfica, com o resultado de sucesso em cada ação e o log completo no painel de logging. Situação atual: disponível.



Tratamento de dados

4.1 Valor sentinela -200

O dataset original usa -200 para indicar ausencia de leitura em qualquer sensor. O passo Null If converte esse valor em nulo antes de qualquer calculo, evitando que ele distorça médias e indicadores.

4.2 Linhas vazias do export

O arquivo termina com 114 linhas totalmente vazias, um artefato do processo de exportação do dataset. Essas linhas são descartadas logo após a leitura do CSV, antes de qualquer outro tratamento.

4.3 Separação de leituras inválidas

Uma leitura é considerada inválida quando os três poluentes de referência (CO, NOx e NO2) estão todos ausentes. Essas leituras são gravadas em `leituras_rejeitadas`, com o motivo da rejeicao, em vez de entrarem no calculo dos indicadores.

4.4 Classificação analítica

As faixas de BOM, MODERADO e CRÍTICO para CO e NO2 são uma classificação analítica definida para este projeto, calculada a partir dos percentis do próprio dataset (a faixa CRÍTICO representa aproximadamente o décimo mais alto de cada poluente). Não há referência a uma norma regulatória externa.



Banco de dados

5.1 Estrutura de tabelas

Tabela / visão	Finalidade
<code>leituras_qualidade_ar</code>	Dados tratados e detalhados, uma linha por leitura horária válida
<code>leituras_rejeitadas</code>	Leituras sem nenhum poluente de referência
<code>resumo_diario_qualidade_ar</code>	Indicadores agregados por dia
<code>resumo_mensal_qualidade_ar</code>	Indicadores agregados por mês
<code>vw_indicadores_gerais</code>	Indicadores gerais do período, usados pelo dashboard
<code>vw_ranking_piores_dias</code>	Os 10 dias de piores condições

O detalhamento completo das colunas está em `docs/pt/contrato-banco.md` no repositório.

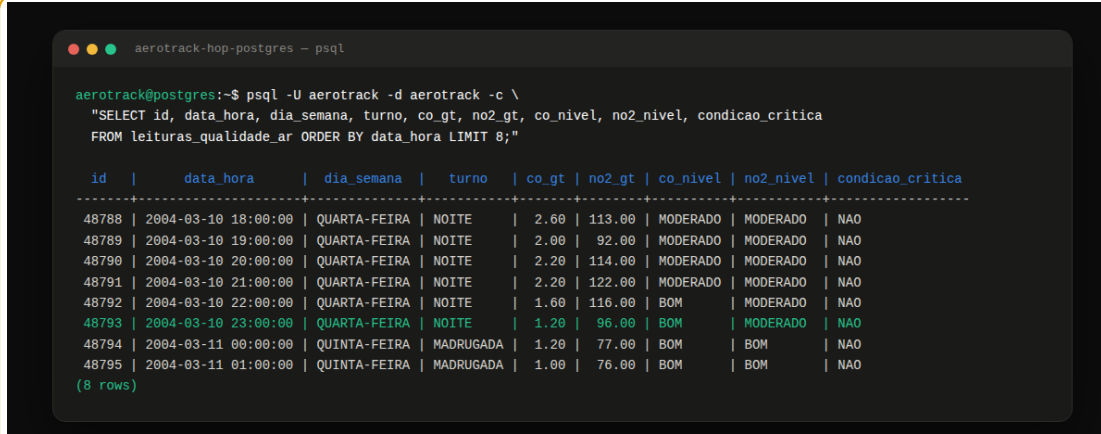
```
aerotrack-hop-postgres ~ psql
aerotrack@postgres:~$ psql -U aerotrack -d aerotrack -c "\dt"

      List of relations
Schema |          Name          | Type | Owner
-----+-----+-----+-----
public | leituras_qualidade_ar  | table | aerotrack
public | leituras_rejeitadas    | table | aerotrack
public | resumo_diario_qualidade_ar | table | aerotrack
public | resumo_mensal_qualidade_ar | table | aerotrack
(4 rows)

aerotrack@postgres:~$ psql -U aerotrack -d aerotrack -c "\dv"

      List of relations
Schema |          Name          | Type | Owner
-----+-----+-----+-----
public | vw_indicadores_gerais  | view  | aerotrack
public | vw_ranking_piores_dias | view  | aerotrack
(2 rows)
```

Figura 5.1: Tabelas e visões criadas no PostgreSQL (`psql`, `\dt` e `\dv`). Situação atual: disponível.



```
aerotrack-hop-postgres - psql

aerotrack@postgres:~$ psql -U aerotrack -d aerotrack -c \
"SELECT id, data_hora, dia_semana, turno, co_gt, no2_gt, co_nivel, no2_nivel, condicao_critica
FROM leituras_qualidade_ar ORDER BY data_hora LIMIT 8;"
```

id	data_hora	dia_semana	turno	co_gt	no2_gt	co_nivel	no2_nivel	condicao_critica
48788	2004-03-10 18:00:00	QUARTA-FEIRA	NOITE	2.00	113.00	MODERADO	MODERADO	NAO
48789	2004-03-10 19:00:00	QUARTA-FEIRA	NOITE	2.00	92.00	MODERADO	MODERADO	NAO
48790	2004-03-10 20:00:00	QUARTA-FEIRA	NOITE	2.20	114.00	MODERADO	MODERADO	NAO
48791	2004-03-10 21:00:00	QUARTA-FEIRA	NOITE	2.20	122.00	MODERADO	MODERADO	NAO
48792	2004-03-10 22:00:00	QUARTA-FEIRA	NOITE	1.60	116.00	BOM	MODERADO	NAO
48793	2004-03-10 23:00:00	QUARTA-FEIRA	NOITE	1.20	96.00	BOM	MODERADO	NAO
48794	2004-03-11 00:00:00	QUINTA-FEIRA	MADRUGADA	1.20	77.00	BOM	BOM	NAO
48795	2004-03-11 01:00:00	QUINTA-FEIRA	MADRUGADA	1.00	76.00	BOM	BOM	NAO

(8 rows)

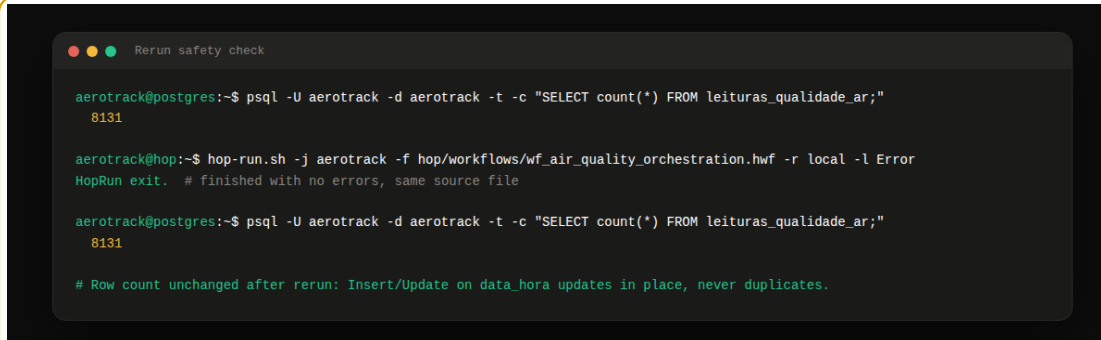
Figura 5.2: Amostra real de `leituras_qualidade_ar`, com a hora 23h já classificada corretamente como NOITE após a correção descrita no registro de ajustes. Situação atual: disponível.

5.2 Reexecução e controle de duplicidade

Pergunta obrigatória do trabalho final

Se o pipeline for executado novamente com a mesma fonte de dados, o que acontece? A tabela `leituras_qualidade_ar` tem restrição `UNIQUE` em `data_hora`, e a carga usa `Insert/Update`: registros existentes são atualizados, nenhuma linha é duplicada. As tabelas de resumo seguem a mesma lógica, com chave em `data_referencia` e em `(ano, mes)`. A tabela de rejeitados é truncada e recarregada por completo a cada execução, pois funciona como um log da última carga.

Esse comportamento foi validado executando o workflow duas vezes seguidas sobre o mesmo arquivo fonte: a contagem de linhas em `leituras_qualidade_ar` permaneceu em 8131 nas duas execuções.



```
Rerun safety check

aerotrack@postgres:~$ psql -U aerotrack -d aerotrack -t -c "SELECT count(*) FROM leituras_qualidade_ar;"
8131

aerotrack@hop:~$ hop-run.sh -j aerotrack -f hop/workflows/wf_air_quality_orchestration.hwf -r local -l Error
HopRun exit. # finished with no errors, same source file

aerotrack@postgres:~$ psql -U aerotrack -d aerotrack -t -c "SELECT count(*) FROM leituras_qualidade_ar;"
8131

# Row count unchanged after rerun: Insert/Update on data_hora updates in place, never duplicates.
```

Figura 5.3: Contagem de linhas antes e depois da reexecução do workflow, comprovando a ausência de duplicidade. Situação atual: disponível.



Indicadores e dashboard

6.1 Indicadores calculados

- Total de leituras tratadas e rejeitadas.
- Percentual de horas em condição crítica.
- CO e NO2 médios e máximos do período.
- Evolução mensal de CO e de NO2.
- Distribuição das leituras por nível (BOM, MODERADO, CRÍTICO).
- Leituras por turno do dia.
- Ranking dos dez dias com piores condições.

6.2 Dashboard

O dashboard é uma página HTML estática que lê os indicadores exportados do PostgreSQL em formato JSON pelo script `scripts/export_dashboard.sh`.

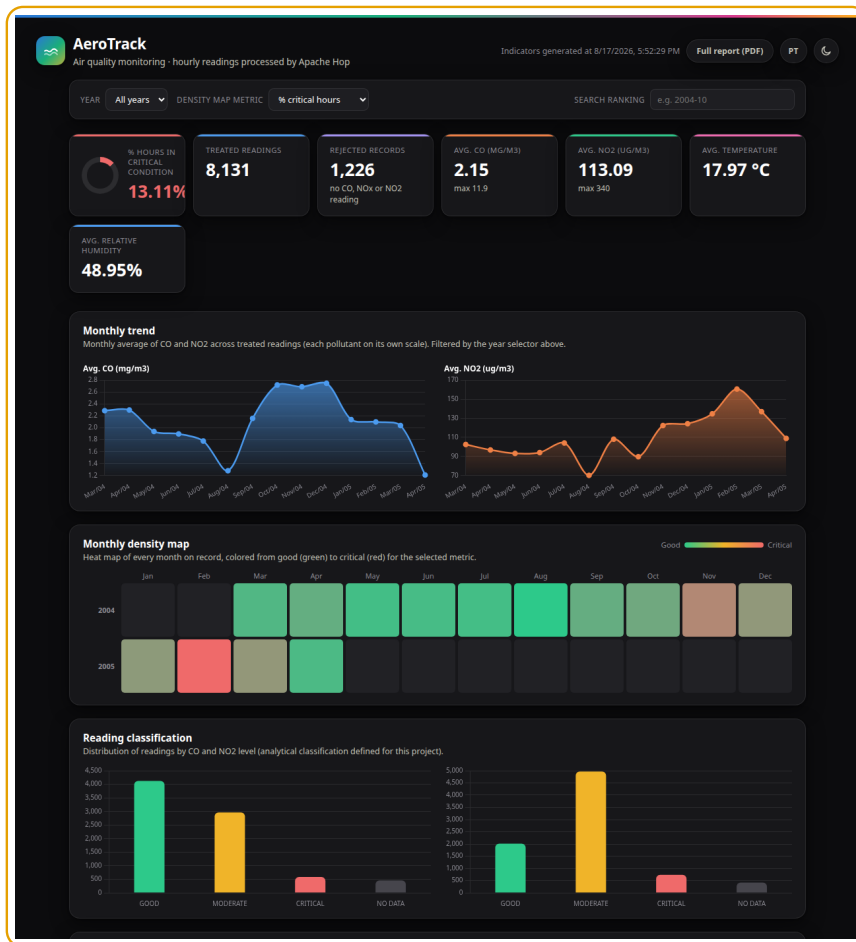


Figura 6.1: Dashboard AeroTrack, cartões de indicadores e gráficos de evolução mensal e classificação. Situação atual: disponível.

Ranking of the 10 worst days				
Days sorted by the highest daily average CO, with average NO2 and percentage of hours in critical condition. Click a column to sort, or use the search box above.				
DATE	AVG. CO	AVG. NO2	CRITICAL HOURS	% CRITICAL HOURS
10/19/2004	5.65	139.75	6	75%
12/23/2004	5.32	166.52	17	70.83%
11/22/2004	5.06	163	15	62.5%
11/25/2004	4.64	159.74	13	54.17%
12/15/2004	4.43	162.96	13	54.17%
11/1/2004	4.33	102.22	11	45.83%
4/14/2004	4.19	130.53	5	33.33%
10/6/2004	4.19	98.38	4	50%
2/10/2005	4.16	223.35	16	66.67%
12/22/2004	4.06	172.09	10	41.67%

Figura 6.2: Ranking dos dez dias com piores condições registradas. Situação atual: disponível.

O dashboard também traz um seletor manual de tema claro e escuro, um mapa de densidade mensal, filtros de ano e de métrica, uma tabela de ranking pesquisável e ordenável, e um painel com link e prévia embutida do relatório de desenvolvimento em PDF.

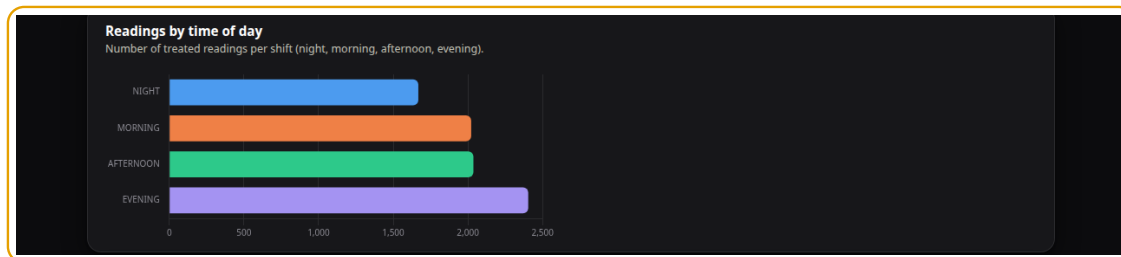


Figura 6.3: Distribuição das leituras por turno do dia. Situação atual: disponível.

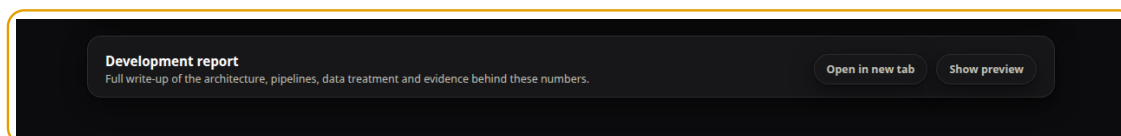


Figura 6.4: Painel do dashboard com o link e a prévia embutida do relatório de desenvolvimento em PDF. Situação atual: disponível.

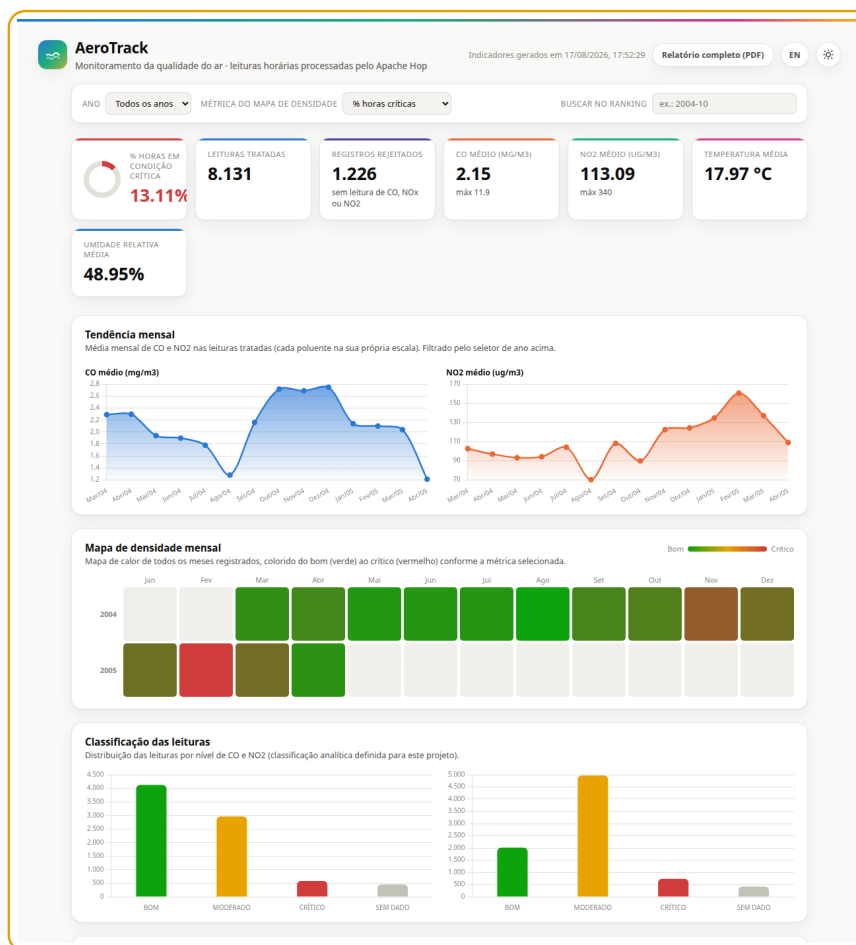


Figura 6.5: Dashboard no tema claro e em português, evidenciando os seletores de tema e idioma. Situação atual: disponível.



Validação dos resultados

7.1 Execução do workflow completo

```
2026/08/17 - wf_air_quality_orchestration - Starting action [Ingestion and treatment]
2026/08/17 - Write rejected reading.0 - Finished processing (I=0, O=1226, R=1226, W=1226, U=0, E=0)
2026/08/17 - Write treated reading.0 - Finished processing (I=8131, O=0, R=8131, W=8131, U=0, E=0)
2026/08/17 - wf_air_quality_orchestration - Finished action [Ingestion and treatment] (result=[true])
2026/08/17 - wf_air_quality_orchestration - Starting action [Indicator consolidation]
2026/08/17 - Write daily summary.0 - Finished processing (I=363, O=0, R=363, W=363, U=0, E=0)
2026/08/17 - Write monthly summary.0 - Finished processing (I=14, O=0, R=14, W=14, U=0, E=0)
2026/08/17 - wf_air_quality_orchestration - Finished action [Indicator consolidation] (result=[true])
2026/08/17 - wf_air_quality_orchestration - Workflow execution finished
```

O log completo está em docs/evidencias/logs/execução-workflow-completo.log.

7.2 Indicadores gerais obtidos

Indicador	Valor
Total de leituras tratadas	8131
Total de leituras rejeitadas	1226
Percentual de horas em condição crítica	13,11%
CO médio (máximo)	2,15 mg/m3 (11,90 mg/m3)
NO2 médio (máximo)	113,09 ug/m3 (340,00 ug/m3)
Temperatura média	17,97 graus Celsius
Umidade relativa média	48,95%
Dias consolidados	363
Meses consolidados	14



7.3 Conferência de contagens

A soma de leituras válidas (8131) e rejeitadas (1226) confere com o total de linhas de dados após a remoção das 114 linhas vazias do export (9357 linhas originais de dados). O ranking dos piores dias mostra média de NO₂ e percentual de horas críticas elevados nas mesmas datas de maior média de CO, o que é coerente com o comportamento esperado de poluentes urbanos.



Conclusão

8.1 Principal resultado

O AeroTrack demonstra o ciclo completo de ETL do trabalho final: leitura de um dataset público, tratamento real dentro do Apache Hop (nulos, linhas inválidas, conversão de tipos, classificação), persistência em banco relacional reexecutável sem duplicidade, orquestração por workflow e indicadores apresentados em um dashboard.

8.2 Principais decisões técnicas

- Combinar ingestão, tratamento, enriquecimento e carga em uma única pipeline (p1_01), e a consolidação em outra (p1_02), o que manteve o projeto com duas pipelines coerentes em vez de várias pipelines pequenas.
- Definir a classificação de CO e NO2 a partir dos percentis reais do dataset, em vez de valores arbitrários, para que o indicador de horas críticas fosse informativo.
- Usar Insert/Update com chave de negócio em todas as tabelas para que o processo seja seguro de reexecutar.

8.3 Aprendizado

A etapa mais trabalhosa não foi desenhar o fluxo, e sim garantir que cada transformação gravasse exatamente os campos esperados, principalmente na combinação entre seleção de campos, conversão de tipos e a gravação da chave no Insert/Update. Cada ajuste foi validado executando a pipeline real contra o banco de dados, não apenas revisando a configuração visualmente.



9

Manual de replicação

Este capítulo descreve como reproduzir o AeroTrack do zero em uma máquina limpa. O mesmo conteúdo, sempre atualizado, está em `docs/manual-replicacao.md` no repositório.

9.1 Pré-requisitos

Ferramenta	Versão mínima	Uso
Docker Engine	24	Executar o PostgreSQL local
Docker Compose	2.20	Orquestrar o container do banco
Java (JDK)	21	Executar o Apache Hop
Apache Hop	2.18	Executar as pipelines e o workflow
Python	3.10	Exportar os indicadores para o dashboard

9.2 Passo a passo

1. Clonar o repositório

```
git clone https://github.com/JulianaBallin/AeroTrack.git
cd AeroTrack
```

2. Subir o banco de dados

```
cd database && docker compose up -d
```

3. Registrar o projeto no Apache Hop (uma vez por máquina)

```
hop-conf.sh -pc -p aerotrack -ph "$(pwd)/hop" -pf project-config.json -pkf
```




4. Executar o processo completo

```
hop-run.sh -j aerotrack \  
-f hop/workflows/wf_air_quality_orchestration.hwf -r local -l Basic
```

5. Gerar e abrir o dashboard

```
scripts/export_dashboard.sh  
cd dashboard && python3 -m http.server 8000
```

Script único

Os passos 2, 4 e 5 estão automatizados em `scripts/replicate.sh`. O script verifica os pré-requisitos, sobe o banco, registra o projeto no Hop se necessário, executa o workflow e exporta o dashboard.

9.3 Solução de problemas

Sintoma	Solução
<code>hop-run.sh</code> não encontra o projeto	Repita o passo de registro do projeto
Erro de conexão com o PostgreSQL	Aguarde o container ficar healthy
Porta 5434 ocupada	Ajuste a porta em <code>database/docker-compose.yml</code> e no arquivo de conexão do Hop
Dashboard em branco no navegador	Sirva a pasta com <code>python3 -m http.server</code> , não abra o arquivo diretamente



Equipe

Integrante	GitHub	Frente
Juliana Ballin Lima	@JulianaBallin	Estrutura do projeto Hop, pipelines, banco de dados, workflow, dashboard e documentação
Allef Oliveira Ramos	@allef-oliveira	Regras de classificação e revisão dos indicadores
Fernanda de Oliveira da Costa	@nanda-costa	Validação do tratamento de dados e do contrato do banco
Pedro Henrique Oliveira Dias	@pedroddias-oss	Testes de reexecução, evidências e apresentação